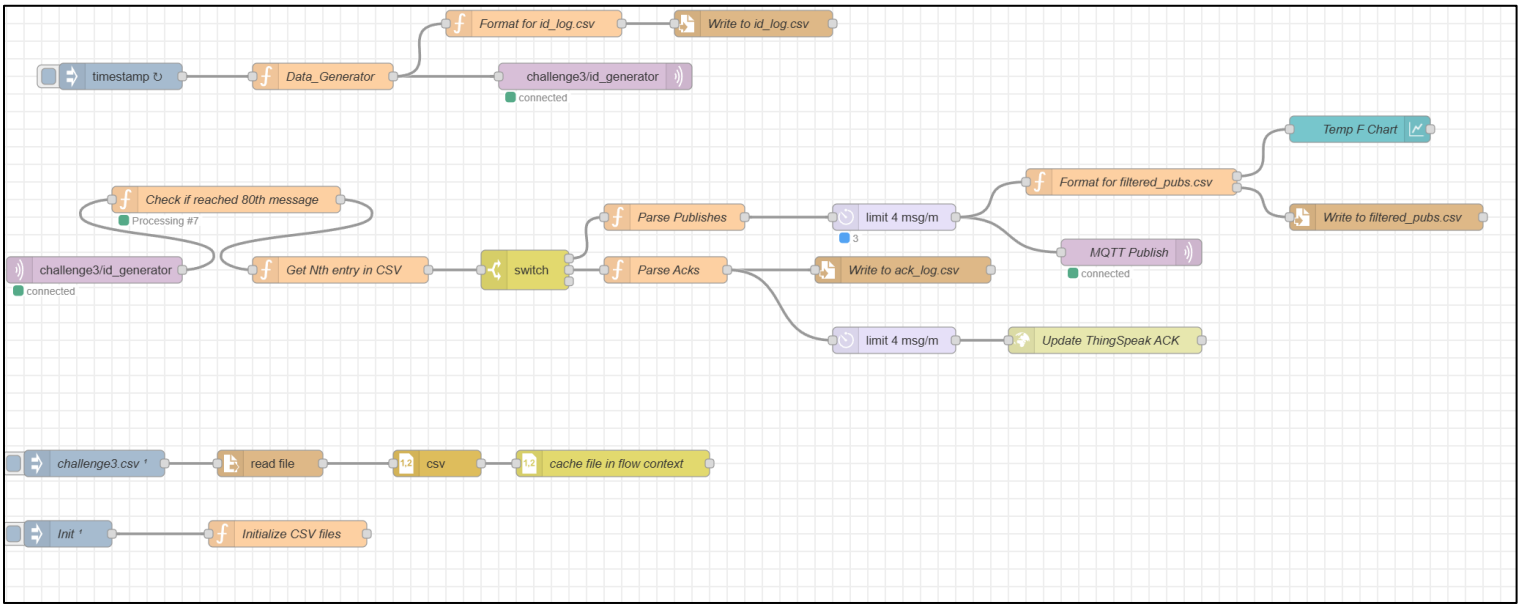
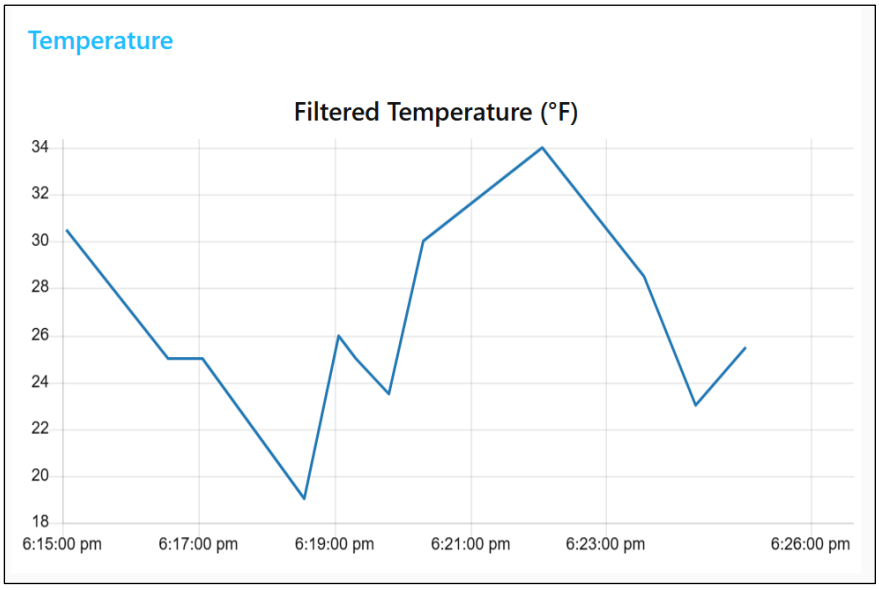
Challenge 3:

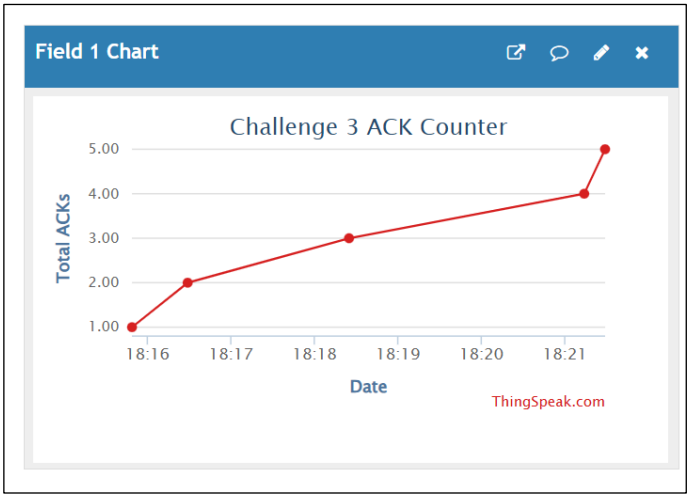
Karim Negm - 11063520

Challenge Flow



Node-red Temperature Chart – ThingSpeak Ack Counter (Channel ID: 2936753)





Explanation

(a) **Top Branch: ID Generation and Logging**

1. **timestamp (Inject Node):**
    - **Meaning:** Triggers the flow periodically.
    - **Function:** Automatically injects a message every 5 seconds to start the process of generating and publishing a new ID.
2. **Data_Generator (Function Node):**
    - **Meaning:** Creates the random ID and timestamp.
    - **Function:** Generates a random integer id (0-30000) and gets the current UNIX timestamp (seconds). It formats these into a JSON string payload like {"id": 12345, "timestamp": 17...}.
3. **Format for id_log.csv (Function Node):**
    - **Meaning:** Prepares a line for the ID log file.
    - **Function:** Takes the id and timestamp generated previously. Gets/increments a row number counter (using context). Formats the output payload as a CSV string: RowNumber,ID,Timestamp\n.
4. **Write to id_log.csv (File Node):**
    - **Meaning:** Saves the generated ID information to a file.
    - **Function:** Appends the CSV string received in its input payload to the id_log.csv file. Runs continuously for every generated ID.
5. **challenge3/id_generator (MQTT Out Node):**
    - **Meaning:** Publishes the generated ID/timestamp message.
    - **Function:** Takes the JSON string payload from Data_Generator and publishes it to the MQTT topic challenge3/id_generator on the local broker (localhost:1884). Runs continuously.

(b) **Middle Branch: Processing Subscribed Messages**

1. **challenge3/id_generator (MQTT In Node):**
    - **Meaning:** Listens for the ID/timestamp messages published by the top branch.
    - **Function:** Subscribes to the challenge3/id_generator topic on the local broker. Outputs the received message payload

2. **Check if reached 80th message (Function Node):**
    - **Meaning:** Enforces the processing limit.
    - **Function:** Counts how many messages have arrived through this input using flow context. If the count is 80 or less, it allows the message to pass through
        - If the count exceeds 80, it returns null, stopping further processing in this branch for that message.
3. **Get Nth entry in CSV (Function Node):**
    - **Meaning:** Retrieves the relevant row from the cached CSV data.
    - **Function:**
        - Gets the value N from the incoming message (id%7711).
        - Retrieves the pre-loaded CSV data array from flow context flow.get("csv_data")
        - Selects the specific row object corresponding to index N from the cached CSV file.
        - Puts this row object into the outgoing message (msg.payload) and passes along (msg.subId)
4. **switch (Switch Node):**
    - **Meaning:** Directs the flow based on the type of message found in the CSV row.
    - **Function:** Inspects a specific field in the incoming payload (col9 field).
        - If it contains "Publish Message", sends the message to Output 1.
        - If it contains "Ack" (e.g., "Connect Ack", "Publish Ack"), sends the message to Output 2.
        - Ignores other message types

(c) **Middle Branch - Publish Path:**

1. **Parse Publishes (Function Node):**
    - **Meaning:** Extracts details needed for re-publishing from the CSV row.
    - **Function:**
        - Takes the CSV row data.
        - Extracts the original topic(s) from the Info/col9 field.
        - Parses the original payload string(s) from the Payload/col10 field into usable object(s).
        - Prepares the data needed for the subsequent MQTT publish and potentially for filtering.

2. **limit 4 msg/m (Delay Node - Publish Path):**
    - **Meaning:** Enforces the rate limit for MQTT publishes.
    - **Function:** Set to "Rate Limit" mode. Allows a maximum of 4 messages per minute to pass through. Queues any excess messages arriving within that minute.
3. **MQTT Publish (MQTT Out Node):**
    - **Meaning:** Sends the constructed publish message based on CSV data.
    - **Function:** Receives the formatted message from the rate limiter (containing the payload specified on Slide 7 and the correct topic). Publishes this message to the local MQTT broker (localhost:1884).

(d) **Middle Branch - ACK Path:**

1. **Parse Acks (Function Node):**
    - **Meaning:** Processes ACK message details and updates counter.
    - **Function:**
        - Takes the CSV row data for an ACK.
        - Increments a global ACK counter.
        - Extracts/cleans the specific ACK type (e.g., "Connect Ack").
        - Prepares the CSV string for ack_log.csv.
        - Adds the current ackCount to the message (msg.currentAckCount). Returns the message.
2. **Write to ack_log.csv (File Node):**
    - **Meaning:** Logs ACK events.
    - **Function:** Appends the formatted CSV string (received from "Parse Acks") to the ack_log.csv file.
3. **limit 4 msg/m (Delay Node - ACK Path):**
    - **Meaning:** Rate limits ThingSpeak updates (optional but recommended).
    - **Function:** Set to "Rate Limit" mode. Allows typically 1 message per 15 seconds (or 4 per minute) to pass. Queues excess messages.
4. **Update ThingSpeak ACK (HTTP Request Node):**
    - **Meaning:** Sends the ACK count to the cloud platform.
    - **Function:** Makes a GET request to the ThingSpeak API URL, inserting the msg.currentAckCount value into the field1 parameter, using the Write API Key.

(e) **Middle Branch - Filter/Chart Path:**

1. **Format for filtered_pubs.csv (Function Node):** (Also acts as Filter)
    - **Meaning:** Filters for Fahrenheit temperature messages and prepares outputs for chart and log file.
    - **Function:** Receives the parsed *original* payload data from "Parse Publishes". Checks if type is "temperature" and unit is "F". If yes:
        - Calculates the mean temperature from the range.
        - Outputs 1: Sends a message containing *only* the numeric mean temperature (msg.payload = meanValue) to the chart.
        - Outputs 2: Increments a filtered log counter. Formats the CSV string (No.,LONG,LAT,...) and sends a message containing this string (msg.payload = csvString) to the file writer.
2. **Temp F Chart (Chart Node):**
    - **Meaning:** Displays Fahrenheit temperature trends.
    - **Function:** Receives messages containing numeric temperature values on its input. Plots these values over time on the Node-RED Dashboard UI.
3. **Write to filtered_pubs.csv (File Node):**
    - **Meaning:** Logs details of the filtered Fahrenheit messages.
    - **Function:** Appends the formatted CSV string (received from output 2 of the filter function) to the filtered_pubs.csv file.

(f) **Bottom Branch: Initialization / Caching**

1. **challenge3.csv (Inject Node):**
    - **Meaning:** Triggers the CSV loading process once on deploy/start.
    - **Function:** Injects a single message shortly after the flow starts.
2. **read file (File In Node):**
    - **Meaning:** Reads the content of the main challenge CSV file.
    - **Function:** Reads the entire challenge3.csv file as a single string.
3. **csv (CSV Node):**
    - **Meaning:** Parses the CSV string into structured data.
    - **Function:** Converts the CSV string into an array of JavaScript objects, using the first row as headers.

4. **cache file in flow context (Function Node):**
    - **Meaning:** Stores the parsed CSV data for later use.
    - **Function:** Takes the array of objects from the csv node (in msg.payload) and saves it into flow context (flow.set("csvData", msg.payload)).
5. **Init (Inject Node):**
    - **Meaning:** Triggers the initialization of log files on deploy/start.
    - **Function:** Injects a single message shortly after the flow starts.
6. **Initialize CSV files (Function Node):**
    - **Meaning:** Creates/overwrites log files with headers.
    - **Function:** Uses the fs module (enabled in settings.js) to write the header lines to id_log.csv, ack_log.csv, and filtered_pubs.csv, ensuring they start fresh on each deploy.